Лабораторна робота №12

Створення FastAPI-додатку з використанням Ormar-моделі без зовнішніх ключів та підключенням до PostgreSQL через Docker.

Хід роботи

1. Підготовка середовища

Перед початком роботи потрібно переконатися, що на комп'ютері встановлено:

- ***Docker*** і ***Docker Compose*** — для створення контейнерів із базою даних і додатком.
- ***Python 3.10+*** (встановлений у системі, використовується в контейнері).

Для зручності всі залежності ми будемо встановлювати всередині Docker-контейнера, тому нічого додатково вручну інсталювати не потрібно.

2. Створення Dockerfile

Файл ***Dockerfile*** описує, як створюється контейнер із нашим FastAPI-додатком.

У ньому ми вказуємо:

- яку версію Python використовувати,
- які пакети встановити (FastAPI, Ormar, SQLAlchemy тощо),
- яку команду запускати при старті контейнера.

Код *Dockerfile*:

```
FROM python:3.10-slim

WORKDIR /app

COPY . /app

RUN pip install --no-cache-dir \
    fastapi==0.95.2 \
    uvicorn==0.23.1 \
    ormar==0.11.2 \
    SQLAlchemy==1.4.38 \
    databases[postgresql]==0.5.5 \
    asyncpg==0.27.0 \
    psycopg2-binary==2.9.9 \
    nest-asyncio==1.6.0

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Пояснення:

- FROM python:3.10-slim — використовуємо мінімальний образ Python 3.10.
- WORKDIR /app — створюємо робочу директорію.
- COPY . /app — копіюємо код у контейнер.
- RUN pip install ... — встановлюємо всі потрібні бібліотеки.
- EXPOSE 8000 — відкриваємо порт, через який можна звертатись до API.
- CMD — команда, що запускає наш FastAPI-сервер при старті контейнера.

3. Налаштування docker-compose.yml

Файл ***docker-compose.yml*** дозволяє одночасно запустити два сервіси:

- базу даних ***PostgreSQL***,
- ***FastAPI-додаток***, який до неї підключається.

Код *docker-compose.yml*:

```yaml
version: "3.9"

services:
  postgres-db:
    image: postgres:15
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
      POSTGRES_DB: booksdb
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

  fastapi-app:
    build: .
    depends_on:
      - postgres-db
    environment:
      DATABASE_URL: "postgresql://user:password@postgres-db:5432/booksdb"
    ports:
      - "8000:8000"
    volumes:
      - .:/app

volumes:
  pgdata:
```

Пояснення:

- Сервіс *postgres-db* створює контейнер із PostgreSQL. У ньому ми задаємо логін, пароль і назву бази.

- Сервіс *fastapi-app* створює контейнер з нашим FastAPI-додатком, який автоматично підключається до бази.
- Через *ports* відкриваються зовнішні порти для доступу до API (8000) і бази (5432).
- *depends_on* гарантує, що база даних буде запущена першою.

Додаткові зауваження:

- Якщо у вас вже встановлено *PostgreSQL* локально, ви можете використовувати локальну базу даних без створення контейнера для неї. У такому випадку в файлі *docker-compose.yml* залиште лише сервіс для FastAPI-додатку (*fastapi-app*), а у змінній середовища *DATABASE_URL* вкажіть ваш локальний шлях до бази.
- Якщо у вас вже запущений локальний *PostgreSQL*, але ви все ж хочете створити окремий контейнер для додатку, зверніть увагу на налаштування портів:
  ```
  ports:
   - "5432:5432"
  ```
  Порт 5432 може бути вже зайнятий локальною базою, тому *Docker* не зможе створити контейнер.
  У такому разі потрібно змінити зовнішній порт контейнера на інший, наприклад:
  ```
  ports:
   - "5433:5432"
  ```
  І не забудьте оновити *DATABASE_URL*, вказавши новий порт:
  ```
  environment:
  DATABASE_URL: "postgresql://user:password@postgres-db:5433/booksdb"
  ```

4. Підключення до бази даних

У файлі ***main.py*** ми створюємо підключення до PostgreSQL і описуємо базові елементи бази даних.

```
import databases
import sqlalchemy
import ormar

DATABASE_URL = os.getenv(
    "DATABASE_URL", "postgresql://user:password@localhost:5432/booksdb"
)
database = databases.Database(DATABASE_URL)
metadata = sqlalchemy.MetaData()
engine = sqlalchemy.create_engine(DATABASE_URL))
```

- DATABASE_URL — адреса підключення до PostgreSQL.
- database — об’єкт для асинхронної взаємодії з базою.

- metadata і engine — потрібні для створення таблиць через SQLAlchemy.

5. Створення моделі даних

Щоб взаємодіяти з таблицею в базі даних, ми створюємо модель **Book**. Модель описує, які поля матиме таблиця та які типи даних у них зберігаються.

```
class Book(ormar.Model):
    class Meta:
        tablename = "books"
        metadata = metadata
        database = database

    id: int = ormar.Integer(primary_key=True)
    title: str = ormar.String(max_length=100)
    description: str = ormar.Text()
    published_year: int = ormar.Integer()
```

- *id* — унікальний ідентифікатор (primary key).
- *title* — назва книги (рядок до 100 символів).
- *description* — короткий опис.
- *published_year* — рік видання.
- *Meta* — службовий клас, який пов’язує модель із базою даних.

6. Ініціалізація FastAPI-додатку

Тепер створимо об’єкт додатку і налаштуємо його:

```
import nest_asyncio
from fastapi import FastAPI, HTTPException
from ormar.exceptions import NoMatch

app = FastAPI(title="Books API")
nest_asyncio.apply()
```

7. Додавання початкових даних (seed)

Щоб таблиця не була порожньою, ми одразу додамо кілька записів.

```
async def seed_data():
    if await Book.objects.count() == 0:
        books_data = [
            {"title": "1984", "description": "Dystopian novel",
"published_year": 1949},
            {"title": "To Kill a Mockingbird", "description": "Classic
novel", "published_year": 1960},
            {"title": "The Great Gatsby", "description": "Novel by F.
Scott Fitzgerald", "published_year": 1925},
        ]
        for b in books_data:
            await Book.objects.create(**b)
        print("Дані додані в базу!")
```

Функція перевіряє, чи таблиця порожня, і додає три книги, якщо даних немає.

8. Події запуску та завершення

Коли FastAPI запускається або завершує роботу — потрібно підключитись до бази та коректно відключитися.

```
@app.on_event("startup")
async def startup():
    metadata.create_all(engine)
    if not database.is_connected:
        await database.connect()
    await seed_data()

@app.on_event("shutdown")
async def shutdown():
    if database.is_connected:
        await database.disconnect()
```

startup() — створює таблиці (якщо їх ще немає) і підключається до бази.

shutdown() — розриває підключення при завершенні роботи додатку.

9. Реалізація API-ендпоїнтів

Тепер створимо два основних маршрути — для перегляду всіх книг і пошуку за ID.

```
@app.get("/books/", tags=["Books"])
async def get_all_books():
    return await Book.objects.all()

@app.get("/books/{book_id}", tags=["Books"])
async def get_book_by_id(book_id: int):
    try:
        book = await Book.objects.get(id=book_id)
        return book
    except NoMatch:
        raise HTTPException(status_code=404, detail="Книга не знайдена")
```

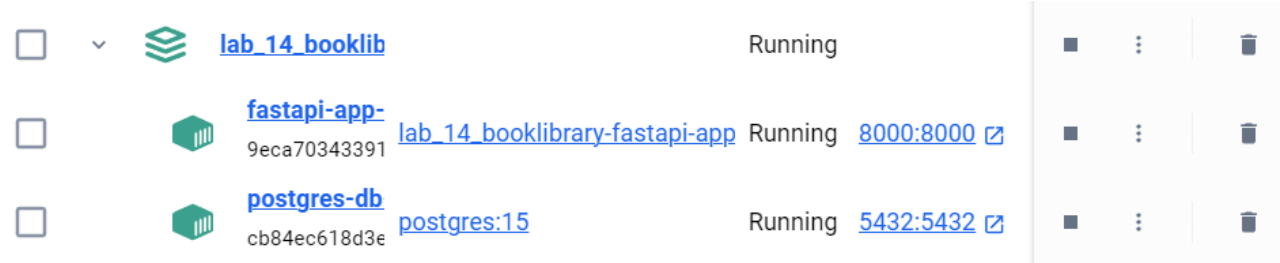
10. Запуск проєкту

У терміналі вводимо:

docker-compose up –build

Docker автоматично створить два контейнери:

- один із базою PostgreSQL,
- інший — із нашим FastAPI-додатком.



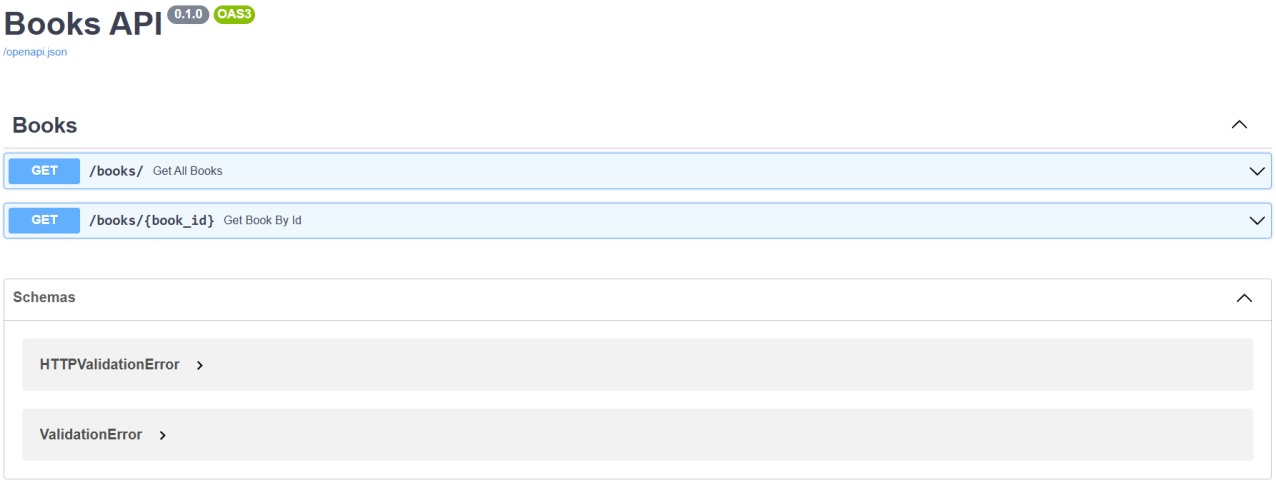
Після запуску переходимо за адресою:

http://localhost:8000/docs

11. Тестування через Swagger

У Swagger ми бачимо два ендпоїнти:

- GET /books/ — показує всі книги.
- GET /books/{id} — шукає конкретну книгу.



При виконанні запиту /books/ ми отримуємо:

